

Credit Card Default Prediction through Small Transformer-based Language Model

IFTE0002

Group D

Contents

1	Introduction	1
2	Data Exploration	2
2.1	Class Distribution	2
2.2	Feature Distributions	2
2.3	Temporal Repayment Patterns	3
2.4	Correlation Analysis	4
2.5	Train / Validation / Test Split	5
3	Small Transformer-based Model Build-up	6
3.1	Tokenisation	6
3.2	Embedding Design	7
3.3	Positional Encoding	8
3.4	Multi-Head Self-Attention	9
3.5	Transformer Block Architecture	10
3.6	Classification Head	10
3.7	Loss Function and Optimiser	11
4	Experiments, Results, and Discussions	12
4.1	Experimental Setup	12
4.2	Small Transformer-based Model Results	13
4.3	Random Forest Results	14
4.4	Model Comparison	16
4.5	Limitations and Improvements	17
5	Conclusions	18
	Acknowledgements	19
	References	20
	Appendices	21

1 Introduction

Credit card default prediction is widely used to develop credit risk management strategies, calculate capital reserve requirements, forecast losses, and determine provisions (Bhandary & Ghosh, 2025). The prediction accuracy of default probability is therefore more valuable than the standard binary classification of good and bad credit card users (Aboagye et al., 2022). Ultimately, it is used to assist financial institutions in determining the appropriateness of providing credit to a client by utilising the financial data to reduce vulnerability (Alam et al., 2020).

Traditional statistical approaches, such as logistic regression and discriminant analysis, are used to build models of risk prediction (Yeh & Lien, 2009). Compared to machine-learning models, they did not give significant results, given the nature of imbalanced datasets (Alam et al., 2020). Hence, this project applies a small transformer-based model built from scratch to the task of credit card default prediction using the UCI Default of Credit Card Clients Dataset (Yeh & Lien, 2009). Since the dataset is tabular, a tokenisation scheme is designed to group the static features into five demographic tokens and the monthly financial behaviour is represented as a sequence of six temporal tokens, allowing the application of a self-attention mechanism. Although tree-based models are frequently cited as the superior choice for tabular data (Shwartz-Ziv & Armon, 2022), the sequential nature of financial history motivates the transformer approach. The primary objective of this project is to compare this architecture against a tuned Random Forest benchmark to evaluate their relative performance on credit default prediction and the suitability of the transformer-based architectures on structured financial data.

The remainder of the report is structured as follows: Section 2 examines key distribution properties, class imbalance, and feature relationships that motivate the following modelling approach. Section 3 details the design and implementation methodology of the small transformed-based language model. Section 4 presents the experimental results for both the Random Forest and transformer-based models, along with a critical discussion of the findings, a comparative analysis, and limitations. Section 5 provides a summary of key findings for the models in credit card default prediction.

2 Data Exploration

The dataset covers 30,000 credit card clients in Taiwan, with records spanning April to September 2005. Once the ID column is removed, each row has 23 input features and a single binary label indicating whether the client defaulted the following month (1 = default, 0 = no default).

2.1 Class Distribution

Figure 1 shows that roughly 78% of clients (23,364) did not default, while the remaining 22% (6,636) did. This kind of imbalance is common in credit risk data (most people pay their bills), but it does make the classification task harder. A model that simply predicts “no default” for everyone would achieve 78% accuracy while being completely useless. For this reason, we focus on F1-score and ROC-AUC rather than raw accuracy when evaluating model performance.

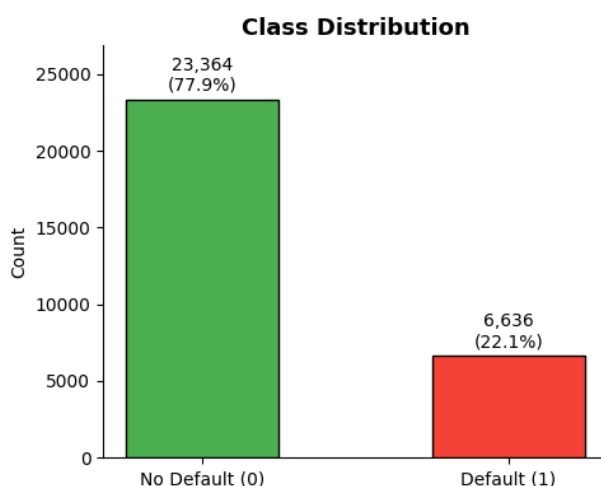


Figure 1: Class distribution of the target variable.

2.2 Feature Distributions

The 23 features fall into four groups: static demographics (LIMIT_BAL, SEX, EDUCATION, MARRIAGE, AGE), monthly repayment status (PAY_0, PAY_2–PAY_6, covering September through April), monthly bill amounts (BILL_AMT1–BILL_AMT6), and monthly payment amounts (PAY_AMT1–PAY_AMT6).

Figure 2 plots three of these features split by default outcome. Clients who defaulted tend to have lower credit limits; this is unsurprising, as higher-risk borrowers are typically given less credit to begin with. Age shows little difference between the two groups. The most telling feature is PAY_0 (September repayment status): defaulters have noticeably higher delay scores, which suggests recent payment behaviour is one of the stronger signals in the data.

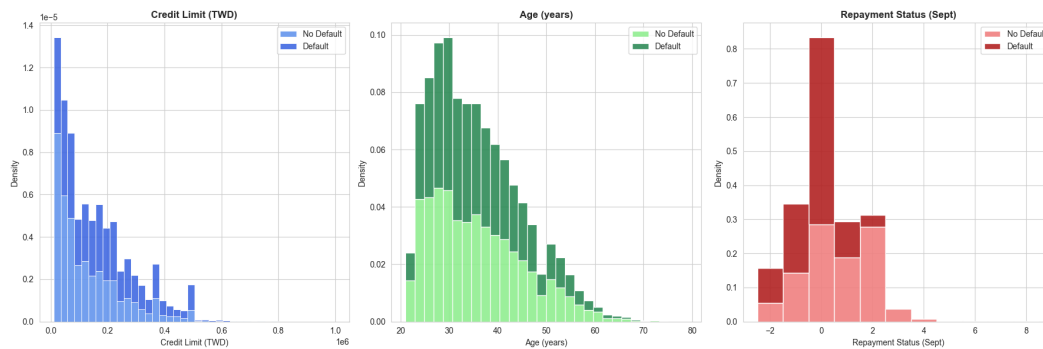


Figure 2: Feature Distributions by Default Status for LIMIT_BAL, AGE, and PAY_0.

2.3 Temporal Repayment Patterns

Figure 3 tracks average repayment delay scores month by month, split by default outcome. The two groups are clearly separated at every time point, with defaulters consistently showing higher delay scores, and the gap widens as we approach September. This pattern suggests that payment behaviour is not just a one-off snapshot but a trajectory, which is part of why we chose a sequence-based model rather than treating each month independently.

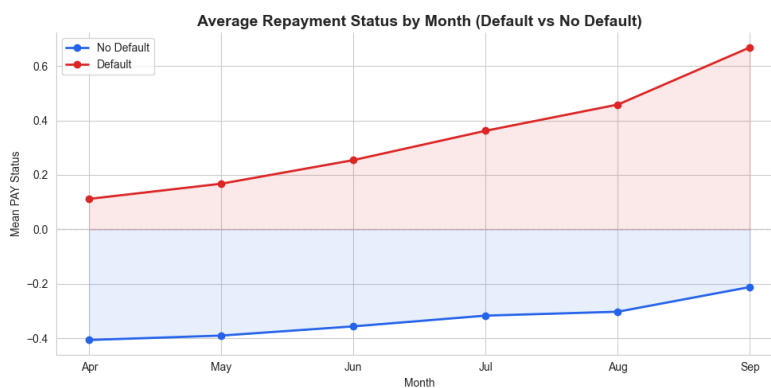


Figure 3: Mean Monthly Repayment Status (PAY)

2.4 Correlation Analysis

Figure 4 shows a correlation matrix for the main features. The PAY_* columns are strongly correlated with each other ($r \approx 0.5$ – 0.8). This relationship is expected, as an individual who was late paying in April is likely to be late again in May. The BILL_AMT columns are even more tightly correlated ($r \approx 0.92$ – 0.95), reflecting the fact that outstanding balances tend to carry over month to month. Looking at the target variable, PAY_0 and PAY_2 have the strongest associations with default, consistent with what Figure 2 already suggested. LIMIT_BAL has a mild negative correlation with the target ($r \approx -0.15$), meaning higher credit limits are slightly linked to lower default rates. The strong inter-correlation among the PAY_* columns directly motivates grouping these features by month in the tokenisation design (Section 3.1), as each monthly token captures these jointly correlated quantities within a single representation.

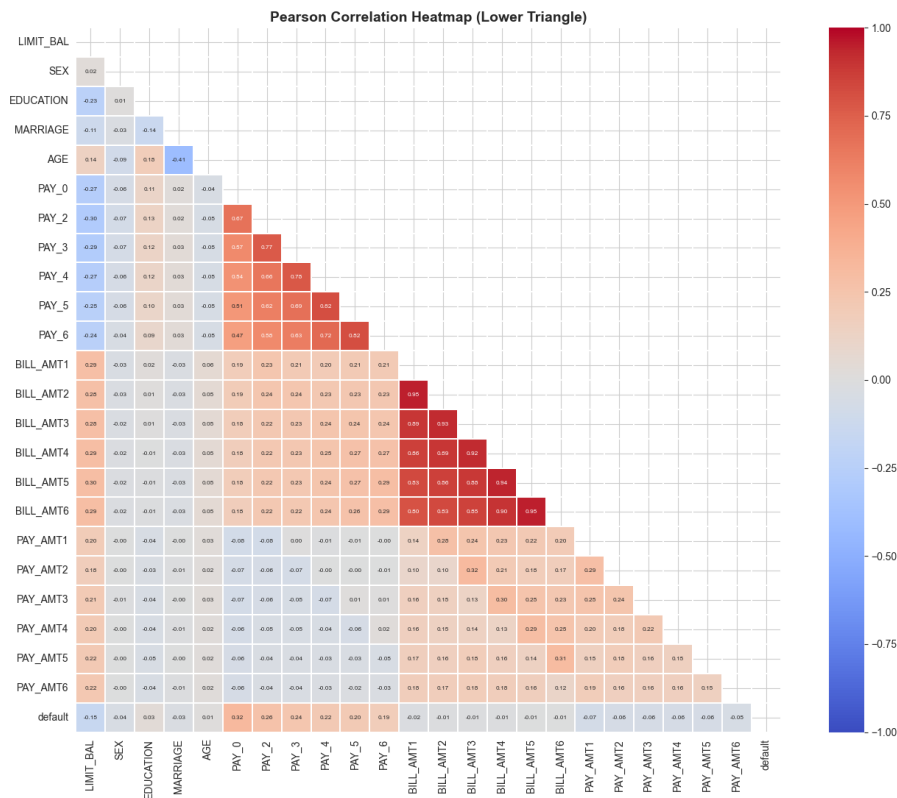


Figure 4: Pearson correlation matrix for selected features and the target variable.

2.5 Train / Validation / Test Split

The data was divided into three sets: 70% for training (21,000 samples), 15% for validation (4,500 samples), and 15% for testing (4,500 samples). All splits used stratified sampling on the target variable to preserve the class ratio in each subset, with a fixed random seed of 42 for reproducibility. The validation set is used during training to monitor performance and avoid overfitting while the test set is held back entirely until final evaluation. Feature scaling was applied after splitting to avoid data leakage. Two `StandardScaler` instances were fitted on the training set only and then applied to all three subsets: one for the static numerical features (`LIMIT_BAL` and `AGE`), and one for the monthly numerical features (`BILL_AMT*`, `PAY_AMT*`).

3 Small Transformer-based Model Build-up

This section describes the encoder-only transformer architecture used for binary classification. We walk through each component in order, from tokenisation and embedding through to the classification head and training setup.

3.1 Tokenisation

A key challenge in applying transformer models to tabular data is the absence of a natural sequential structure. Unlike text, where tokens correspond to words, a tabular record contains heterogeneous features measured at different scales and with different semantics. The tokenisation strategy adopted here is motivated by the temporal structure of the dataset: six months of financial behaviour form a natural sequence, whilst static demographic features constitute a shared context for the entire sequence.

Each customer record is converted into a sequence of 11 tokens: five static demographic tokens followed by six monthly temporal tokens, ordered from oldest to most recent.

- **Tokens S1–S5 Static Demographic Tokens:** Each of the five time-invariant features is represented as its own individual token: credit limit (LIMIT_BAL), sex (SEX), education level (EDUCATION), marital status (MARRIAGE), and age (AGE). Giving each feature a dedicated token allows the attention mechanism to learn which specific demographic attributes are most relevant for predicting default, rather than collapsing all static information into a single undifferentiated vector.
- **Tokens T1–T6 Monthly Temporal Tokens:** Each token corresponds to one calendar month from April (T1, oldest) through September (T6, most recent, 2005), encoding three quantities for that month: repayment status (PAY_*), bill amount (BILL_AMT*), and actual payment amount (PAY_AMT*). PAY_* values are shifted by +2 before embedding to ensure all indices are non-negative.

The full sequence is therefore ordered as $[S1, S2, S3, S4, S5, T1, T2, T3, T4, T5, T6]$. Self-attention operates over all eleven positions simultaneously, allowing the model to discover dependencies both across months (e.g., whether a pattern of delays in earlier months predicts acceleration toward September) and between static context

and temporal behaviour (e.g., a low credit limit amplifying the risk signal from repeated delays). Figure 5 illustrates the tokenisation of Customer ID 1 ($\text{LIMIT_BAL} = 20,000$, $\text{AGE} = 24$) into this 11-token sequence, showing the five static tokens on the left and the six monthly tokens ordered from oldest to most recent on the right.

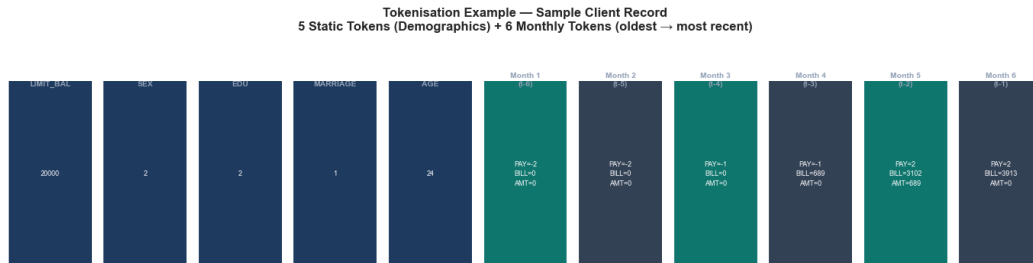


Figure 5: Tokenisation of a Sample Client Record

3.2 Embedding Design

Each token is projected to a 64-dimensional vector through the `CreditTokenEmbedding` layer. Because the features differ in type and scale, each is handled by its own dedicated projection.

Static Token Embeddings ($S1-S5$)

- **Numerical Components** (LIMIT_BAL ($S1$), AGE ($S5$)): The standardised credit limit and age scalar are projected separately to a 64-dimensional vector via a $\text{Linear}(1 \rightarrow 64)$ layer, producing one token for each.
- **Categorical Components** (SEX ($S2$), EDUCATION ($S3$), MARRIAGE ($S4$)): Each categorical feature is mapped to a 64-dimensional vector through its own dedicated Embedding layer (vocabulary sizes 4, 5, and 4 respectively), producing one token each.

Each of the five static features therefore contributes exactly one 64-dimensional token to the sequence. Keeping these features as separate tokens rather than concatenating them into one is a deliberate choice. It allows each head in the multi-head attention to form independent relationships between.

Monthly Token Embeddings ($T1-T6$)

- **Numerical Components** ($BILL_AMT*$, PAY_AMT*): For each month, the scaled bill amount and scaled payment amount are stacked into a 2-dimensional vector and projected to 64 dimensions via a shared $Linear(2 \rightarrow 64)$ layer.
- **Repayment Status Component** (PAY_*): The repayment status value (shifted by $+2$ to ensure a non-negative index) is passed through an $Embedding(16, 64)$ layer, producing a second 64-dimensional vector.
- **Fusion**: The two 64-dimensional vectors are concatenated into a 128-dimensional vector, then compressed back to 64 dimensions via a $Linear(128 \rightarrow 64)$ fusion layer. This concatenation-then-projection design allows the model to learn interactions between the numerical and categorical components of each month before combining them, unlike simple summation which treats the two contributions as independent additive shifts.

This produces one 64-dimensional token per month, giving six monthly tokens in total. All projections and embeddings output vectors of dimension $d_{\text{model}} = 64$, ensuring every token enters the transformer stack in a common representational space.

3.3 Positional Encoding

A learned positional encoding is added to the six monthly tokens ($T1-T6$) only. The five static tokens ($S1-S5$) receive no positional encoding, since their order in the sequence is arbitrary and carries no temporal meaning. And because self-attention is permutation-invariant, applying positional encoding to static tokens would introduce a spurious ordering signal that could mislead the model.

For the monthly tokens, temporal order is meaningful: $T1$ corresponds to the oldest observation (April) and $T6$ to the most recent (September), and the model should be aware of this ordering. A learnable position embedding table $Embedding(6, 64)$ is used, containing one trainable 64-dimensional vector for each of the six monthly positions. During the forward pass, positions 0 through 5 are used to index this table, and the resulting vectors are added element-wise to the corresponding monthly token embeddings:

$$\text{monthly_token}[t] = \text{embedding}(t) + \text{pos_emb}(t), \quad t = 0, 1, \dots, 5.$$

Learned positional embeddings are preferred over fixed sinusoidal encodings here because the monthly sequence length is fixed at exactly six steps throughout training and inference. With a known, short sequence, a learnable table can optimally encode ordinal position without the generalisation constraints that motivate sinusoidal encodings in variable-length text models. The positional encoding is applied after the token embedding layer and before the first transformer block.

3.4 Multi-Head Self-Attention

The attention mechanism is the core of the transformer (Vaswani et al., 2017). Given the token matrix $X \in \mathbb{R}^{11 \times 64}$, comprising five static tokens and six monthly tokens, three linear projections produce query, key, and value matrices:

$$Q = X \cdot W_Q, \quad K = X \cdot W_K, \quad V = X \cdot W_V,$$

$$\text{where } W_Q, W_K, W_V \in \mathbb{R}^{64 \times 64}$$

These are split across 4 attention heads, each working in a 16-dimensional subspace. For each head h , scaled dot-product attention is computed as:

$$\text{Attention}(Q_h, K_h, V_h) = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{16}}\right) V_h.$$

Dividing by $\sqrt{16}$ prevents the attention scores from growing too large and saturating the softmax. The four head outputs are concatenated and projected back through a linear layer $W_O \in \mathbb{R}^{64 \times 64}$:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_4) W_O.$$

Using multiple heads allows the model to capture qualitatively different relationships simultaneously. For example, one head might learn to relate static tokens such as LIMIT_BAL or MARRIAGE to recent monthly payment behaviour (static-to-temporal), while another tracks month-to-month consistency across $T1$ – $T6$ (temporal-to-temporal), and another focuses on recency weighting by attending more to $T6$ than $T1$. Rather than being prescribed, these patterns emerge from training via gradient descent. The attention weight matrices per block are of shape [batch, 4, 11, 11], one entry for each pair of tokens per head, and the 11×11 structure is directly visible in the attention heatmap (Figure 6).

3.5 Transformer Block Architecture

Each transformer block has two sub-layers, attention and feed-forward. Each preceded by layer normalisation (Pre-Norm), followed by residual addition:

$$\begin{aligned}x' &= x + \text{Dropout}(\text{MultiHeadAttn}(\text{LayerNorm}(x))) \\x'' &= x' + \text{Dropout}(\text{FFN}(\text{LayerNorm}(x')))\end{aligned}$$

The feed-forward sub-layer applies two linear transformations with a ReLU in between:

$$\begin{aligned}\text{FFN}(x) &= \text{Dropout}(\text{ReLU}(x \cdot W_1 + b_1)) \cdot W_2 + b_2, \\ \text{where } W_1 &\in \mathbb{R}^{64 \times 64}, \quad W_2 \in \mathbb{R}^{64 \times 64}\end{aligned}$$

The feed-forward hidden dimension equals d_{model} (expansion ratio of 1), as selected by hyperparameter search. Layer normalisation stabilises training, and residual connections help gradients flow through the stack. Dropout of 0.1 is applied in both sub-layers and feed-forward. Two transformer blocks were selected by the hyperparameter search (values tested: 2, 3). For a short 11-token sequence, two blocks are sufficient: the first block identifies which tokens are most relevant, and the second refines those representations by attending to the contextualised output of the first, as is typical in stacked transformer architectures. Deeper stacking provides greater benefit for longer sequences where multi-hop reasoning is needed.

3.6 Classification Head

After both transformer blocks, the output is an 11×64 matrix of contextualised token representations. Mean pooling aggregates these into a single 64-dimensional summary vector:

$$z = \frac{1}{11} \sum_{i=0}^{10} x''_i, \quad \text{for } i = 0, 1, \dots, 10$$

The alternative is a CLS token, prepending a learnable classification token and reading its output after attention. CLS pooling can be advantageous when one token naturally summarises the sequence. Here, no single token plays that role: both static demographics and all six monthly tokens contribute relevant signals. Mean pooling

is therefore more appropriate, ensuring no position is privileged during aggregation. The pooled vector passes through a two-layer head:

$$P(\text{default}) = \sigma(\text{Linear}(32 \rightarrow 1)(\text{ReLU}(\text{Linear}(64 \rightarrow 32)(z))))$$

The model is modest in scale, though the full RandomizedSearchCV (20 iterations $\times$ 3 folds = 60 training runs) takes approximately 2 hours, reflecting the cumulative cost of repeated forward passes over 21,000 training samples per fold rather than model complexity itself.

3.7 Loss Function and Optimiser

Loss Function

Binary Cross-Entropy with Logits (BCEWithLogitsLoss) combines a sigmoid activation with binary cross-entropy in a single numerically stable operation, avoiding floating-point underflow that can occur when sigmoid and BCE are applied separately:

$$\mathcal{L} = -(y \cdot \log \sigma(\text{logit}) + (1 - y) \cdot \log(1 - \sigma(\text{logit})))$$

To address the class imbalance (approximately 3.5 non-default cases per default), a `pos_weight` scalar equal to the training-set ratio of negatives to positives is passed to the loss function, increasing the penalty for missed defaults proportionally.

Optimiser

Adam was used with a fixed learning rate of 1×10^{-3} . Adam maintains per-parameter adaptive learning rates using first and second moment estimates of gradients, making it well-suited to models with heterogeneous parameter types such as embedding layers and linear projections. Gradient norms are clipped at 1.0 per step to prevent occasional large updates from destabilising training.

4 Experiments, Results, and Discussions

4.1 Experimental Setup

Before training, a `NeuralNetClassifier` wrapper was used with a maximum of 50 epochs and a batch size of 128, utilising the GPU where available. Also, a fixed random seed was set for reproducibility. To prevent the model from overfitting and maintain stability, callbacks were implemented to stop training when the validation loss ceased to improve and control the magnitude of the gradients.

Furthermore, we integrated `RandomizedSearchCV` into the training process to tune the model’s inherent hyperparameters and identify the best results. Table 1 demonstrates the grid parameters included. By setting the random search to 20 iterations with 3-fold cross-validation, the total number of training sessions is 60 (plus one final refit session). Regarding the scoring method, the macro-averaged F1-score was selected over ROC-AUC to better address the class imbalance between default and non-default observations.

Table 1: Grid Hyperparameters - Small Transformer-based Model

Hyperparameters	Values
Model Dimensions (d_model) & Heads (n_heads)	(32 & 2), (64 & 4)
Layers (n_layers)	2, 3
Feed Forward Dimensions (d_ff)	64, 128
Dropout Rate (dropout)	0.1, 0.2
Learning Rate (lr)	0.001, 0.0001

To provide a comparison, a Random Forest model was established as a benchmark, utilizing the dataset in its original tabular form. To maintain comparability across architectures, the Random Forest model used the same data partitions as the transformer, with the training and validation sets merged for the Random Forest’s training phase. Hyperparameter tuning was conducted using `GridSearchCV` with 5-fold cross-validation across 27 combinations (Table 2). The best model was selected based on macro-averaged F1-score and refitted on the full training set before evaluation on the held-out test set.

In addition to the original tabular data, a supplementary experiment using tokenised data was conducted for comparison. This model was built with the same settings but utilizes flattened tokenised data for the experiment.

Table 2: Grid Hyperparameters - Random Forest Model

Hyperparameters	Values
Trees (n_estimator)	75, 100, 125
Maximum Depth (max_depth)	15, 20, 25
Minimum Samples per Split (min_samples_split)	8, 10, 12

4.2 Small Transformer-based Model Results

After the training and tuning process, the results show that the optimal model hyperparameters are:

- Model dimension (d_model): 64
- Heads (n_heads): 4
- Layers (n_layers): 2
- Feed forward dimension (d_ff): 64
- Dropout rate (dropout): 0.1
- Learning rate (lr): 0.001

With these hyperparameters, the best cross-validation macro F1-score was 0.69, demonstrating strong prediction performance on the training dataset. Consequently, the model was applied to the test dataset. Table 3 demonstrates the classification report of the final results. The macro average F1-score of 0.68 is only slightly lower than the training result, suggesting that the model is not substantially overfitting. However, the training loss curve (see Appendix) shows that validation loss plateaus after approximately two epochs and oscillates slightly thereafter while training loss continues to fall, indicating a modest generalisation gap that early stopping partially mitigates.

In addition, we utilized the model with a more conservative view by setting the classification threshold to 0.3. Table 4 shows the resulting classification report. Compared to the original 0.5 threshold, the macro average F1-score of the conservative prediction decreased. However, the recall for default cases significantly increased. This aligns with real-world demands to avoid lending to customers who might default.

Table 3: Classification Report - Small Transformer-based Model (Threshold = 0.5)

	Precision	Recall	F1-score	Support
No Default	0.88	0.78	0.83	3504
Default	0.45	0.64	0.53	996
Accuracy			0.75	4500
Macro Average	0.67	0.71	0.68	4500
Weighted Average	0.79	0.75	0.76	4500
ROC-AUC			0.78	

Table 4: Classification Report - Small Transformer-based Model (Threshold = 0.3)

	Precision	Recall	F1-score	Support
No Default	0.93	0.38	0.54	3504
Default	0.29	0.90	0.44	996
Accuracy			0.50	4500
Macro Average	0.61	0.64	0.49	4500
Weighted Average	0.79	0.50	0.52	4500

Through plotting graphs of the model, we acquired a clearer understanding of its progress and decision-making process. Figure 6 demonstrates the attention map for each set of tokens, illustrating which keys (X-axis) each query (Y-axis) attends to. A significant concentration is observed in the Month 6 key, which corresponds to the most recent month prior to the observation. All queries assign substantial attention to this token, suggesting the model has learned that recent behaviour is the most predictive of default. Among the static tokens, the MARRIAGE query shows the highest weights at 0.43, implying a specific correlation the model finds useful for prediction. Conversely, older months receive relatively lower attention weights, indicating they contribute less to the final prediction.

4.3 Random Forest Results

After the training and tuning process, the results show that the optimal hyperparameters of Random Forest model with tabular data are:

- Trees (n_estimator): 125
- Maximum Depth (max_depth): 20
- Minimum Samples per Split (min_samples_split): 10

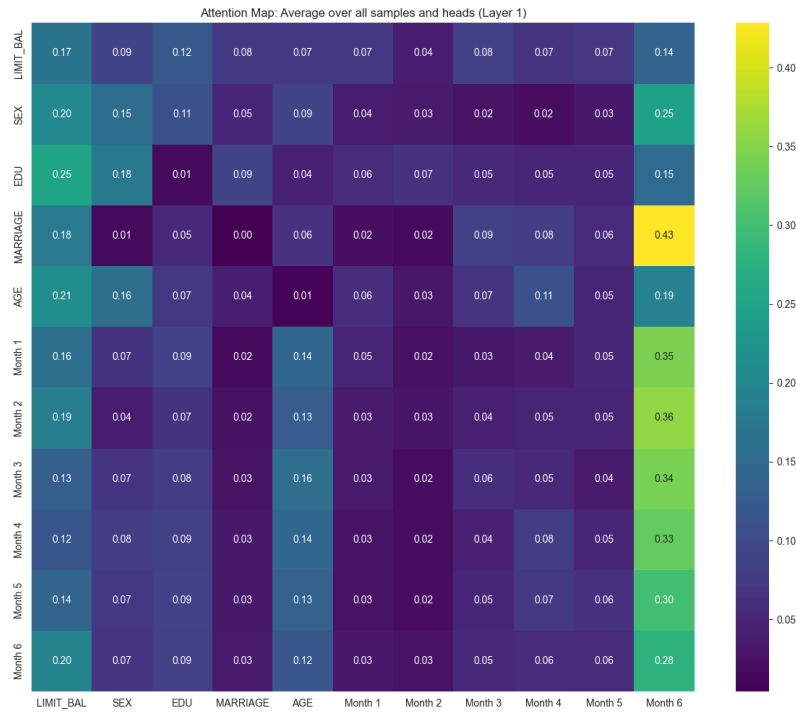


Figure 6: Attention map - Small Transformer-based Model.

As shown in Table 5, the tuned Random Forest achieved an accuracy of 0.82 and a macro F1-score of 0.67. The model was considerably better at identifying non-default customers than defaulters. In particular, recall for the default class remained relatively low, indicating that a substantial number of risky customers were still missed. This represents a significant limitation in a credit-risk setting, where false negatives (missed defaults) are often more costly than false positives.

Table 5: Classification Report - Random Forest Model (Tabular Data)

	Precision	Recall	F1-score	Support
No Default	0.84	0.95	0.89	3504
Default	0.65	0.35	0.46	996
Accuracy			0.82	4500
Macro Average	0.74	0.65	0.67	4500
Weighted Average	0.80	0.82	0.79	4500
ROC-AUC			0.77	

Feature importance analysis provides further insight into the model's behaviour. As illustrated in Figure 7, the most influential predictor was PAY_0, followed by PAY_2, which is financially intuitive. Recent repayment status is naturally a strong indicator of future default, suggesting that the Random Forest model is learning meaningful behavioural risk patterns from the original tabular data.

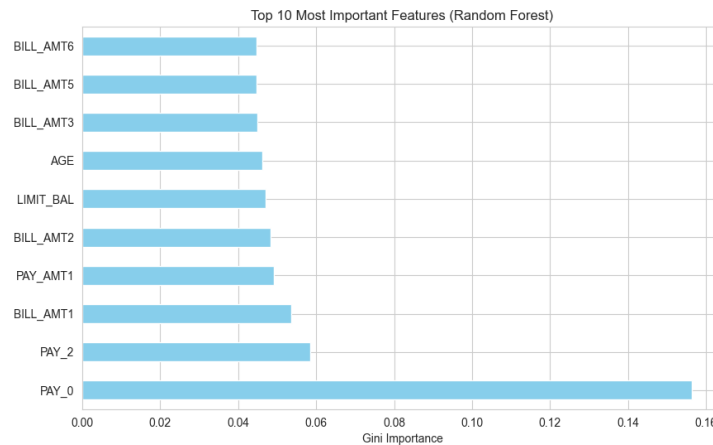


Figure 7: Feature importance - Random Forest.

Finally, the Random Forest using tokenised data did not produce strong performance. As shown in Table 6, although it achieved an accuracy of 0.78, the F1-score for the default class was only 0.01. This indicates the model is essentially predicting the majority class (non-default) for nearly all instances. Therefore, this version will not be included as a primary benchmark in the subsequent discussion.

Table 6: Classification Report - Random Forest Model (Tokenised Data)

	Precision	Recall	F1-score	Support
No Default	0.78	1.00	0.87	3504
Default	0.30	0.00	0.01	996
Accuracy			0.78	4500
Macro Average	0.54	0.50	0.44	4500
Weighted Average	0.67	0.78	0.68	4500

4.4 Model Comparison

Comparing the small transformer-based model to Random Forest model, although their results look similar at first glance, a deeper dive reveals significant differences. From a comprehensive perspective, both models demonstrate similar results. For instance, they both achieved an around 0.68 macro average F1-score, and the small transformer-based model's accuracy of 0.75 is comparable to Random Forest's 0.82. However, looking into the details tells a completely different story. The most important metric is the default recall, where the small transformer-based model reached 0.64 while the Random Forest only achieved 0.35. Two mechanisms explain

this advantage. First, the weighted BCE loss ($\text{pos_weight} \approx 3.5$) directly amplifies the gradient signal for missed defaults during training, whereas the Random Forest receives no equivalent minority-class penalty in its splitting criterion. Second, as evidenced by the attention map in Figure 6, the model learned to concentrate attention on the most recent repayment Month 6, which is the strongest predictor of default. The Random Forest model recency implicitly through feature importance rather than through an explicit learned weighting across the temporal sequence. Furthermore, if a conservative threshold of 0.3 is applied, the recall increases to 0.90. This threshold analysis was applied only to the small transformer-based model, an equivalent Random Forest threshold would similarly boost its recall. Nevertheless, the small transformer-based model’s higher baseline recall at the standard 0.5 threshold reflects a genuine architectural advantage. As a result, the small transformer-based model is the better model for this task due to its high default recall.

4.5 Limitations and Improvements

Despite producing competitive results, the small transformer-based model still presents several limitations. While a weighted BCE loss with a dynamically computed positive class weight of 3.5 (ratio of negative to positive samples: 23,364 / 6,636) was utilized, the model achieved a default recall of only 0.64. This indicates that over a third of true defaulters were missed, suggesting that class imbalance was not fully resolved through loss weighting alone. Furthermore, the temporal tokenisation scheme, while motivated, represents a workaround rather than a natural fit for the transformer architecture, which was originally designed for sequential language data. The monthly grouping imposes a fixed structure that may not fully capture the complex, non-linear interactions inherent in tabular credit data.

Several architectural improvements could significantly strengthen the small transformer-based model in the future. Specifically, a feature-level tokenisation scheme could be adopted, in which each individual feature is embedded as a separate token rather than grouped by month. An approach similar to the FT-Transformer (Huang et al., 2020), which has shown strong performance on tabular benchmarks. This enables the self-attention mechanism to directly model pairwise interactions between individual features, providing a more theoretically grounded representation. Furthermore, transitioning to a more efficient tuning technique, such as Bayesian optimisation, would allow for a more effective exploration of the model’s configuration space compared to standard grid search.

5 Conclusions

The project compared a custom small transformer-based model against a tuned Random Forest benchmark for credit card default prediction on 30,000 client records.

From a broad perspective, both models produced a macro average F1-score of around 0.68, with overall accuracy levels of 0.75 for the small transformer-based model and 0.82 for the Random Forest. However, a closer examination reveals a meaningful difference in default recall, where the small transformer-based model achieved 0.64 compared to only 0.35 for the Random Forest. In a credit risk context, false negatives carry significant financial consequences, making the small transformer-based model's superior recall particularly valuable. This advantage is further reinforced when a threshold of 0.3 is applied, raising the model's recall to 0.90. Consequently, the small transformer-based model is the stronger model for this specific task.

Regarding suitability, the transformer architecture was originally proposed for sequential language modelling (Vaswani et al., 2017), and its inductive biases are less naturally aligned with structured tabular data. Shwartz-Ziv & Armon (2022) further show that tree-based models often outperform deep learning approaches on tabular datasets, a finding reflected here by the Random Forest's higher overall accuracy. Nevertheless, the temporal tokenisation scheme adopted in this study allowed the attention mechanism to reason meaningfully across payment histories, as evidenced by the attention maps, suggesting that with careful tokenisation design, transformer-based models are a viable approach for structured financial data.

Overall, this project demonstrates that with careful tokenisation design, a small transformer-based model is a genuinely viable choice for tabular financial data. While the comparative impact of class weighting strategies across these architectures requires further investigation, the substantial recall advantage convincingly justifies the added architectural complexity over a well-tuned Random Forest model.

Acknowledgements

This project is assisted by LLM models, specifically:

- **Gemini:** Report formatting, Information inquiry
- **Claude:** Code reviewing

Name	Student ID	Key responsibility
Ching Hei Chloe Keung	25198689	Small transformer-based model tuning; Introduction; Limitations & Improvements; References
Ma Suet Nam	25056126	Random Forest model building; Random Forest results
Ryan Lin	24038712	Data exploration; Report formatting
Ryan Shen	25151883	Small transformer-based model building; Experiments & Results; Report formatting
Showgath Miah	25227410	Small transformer-based model building; Data exploration
Ying Shek Isaac Ho	25243807	Small transformer-based model building; Experiments & Results
Yu-Cheng Lin	25166243	Data exploration; Report formatting
Yu-Ling Chen	24107908	Data exploration; Report formatting
Yu-Wei Lin	25063867	Small transformer-based model tuning; Conclusion; Limitations & Improvements; References

References

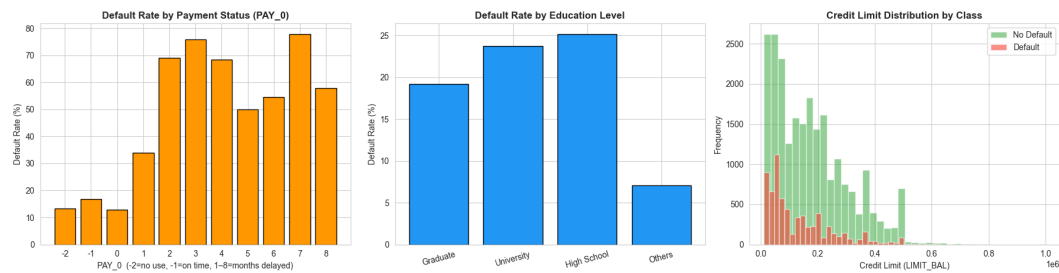
- Aboagye, E. F., Borketey, B., Danquah, K., & Borketey, D. (2022). A predictive modeling approach for optimal prediction of the probability of credit card default. *International Research Journal of Modernization in Engineering Technology and Science*, 4(8). <https://doi.org/10.56726/irjmetS29286>
- Alam, T. M., Shaukat, K., Hameed, I. A., Luo, S., Sarwar, M. U., Shabbir, S., Li, J., & Khushi, M. (2020). An Investigation of Credit Card Default Prediction in the Imbalanced Datasets. *IEEE Access*, 8, 201173–201198. <https://doi.org/10.1109/access.2020.3033784>
- Bhandary, R., & Ghosh, B. K. (2025). Credit Card Default Prediction: An Empirical Analysis on Predictive Performance Using Statistical and Machine Learning Methods. *Journal of Risk and Financial Management*, 18(1), 23. <https://doi.org/10.3390/jrfm18010023>
- Huang, X., Khetan, A., Cvitkovic, M., & Karnin, Z. (2020). TabTransformer: Tabular Data Modeling Using Contextual Embeddings. *arXiv preprint arXiv:2012.06678*. <https://doi.org/10.48550/arxiv.2012.06678>
- Shwartz-Ziv, R., & Armon, A. (2022). Tabular data: Deep learning is not all you need. *Information Fusion*, 81, 84–90. <https://doi.org/10.1016/j.inffus.2021.11.011>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30. <https://doi.org/10.48550/arXiv.1706.03762>
- Yeh, I-Cheng., & Lien, C. (2009). The comparisons of data mining techniques for the predictive accuracy of probability of default of credit card clients. *Expert Systems with Applications*, 36(2), 2473–2480. <https://doi.org/10.1016/j.eswa.2007.12.020>

Appendices

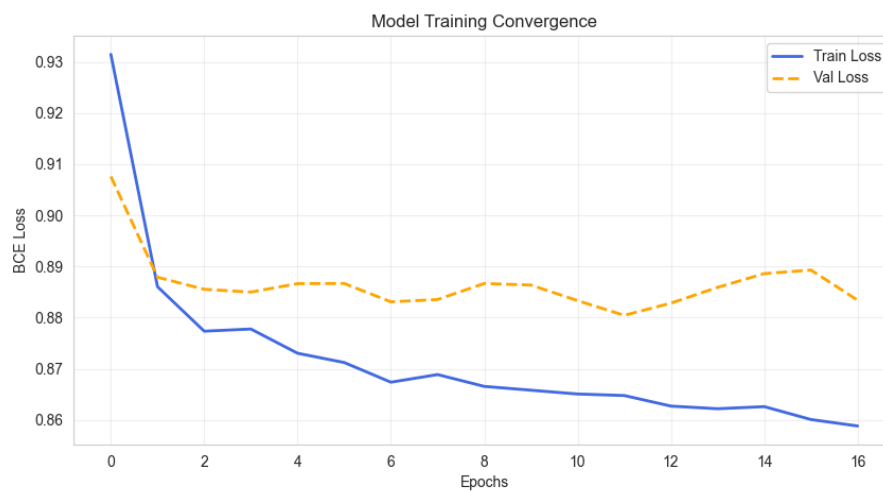
Github Link:

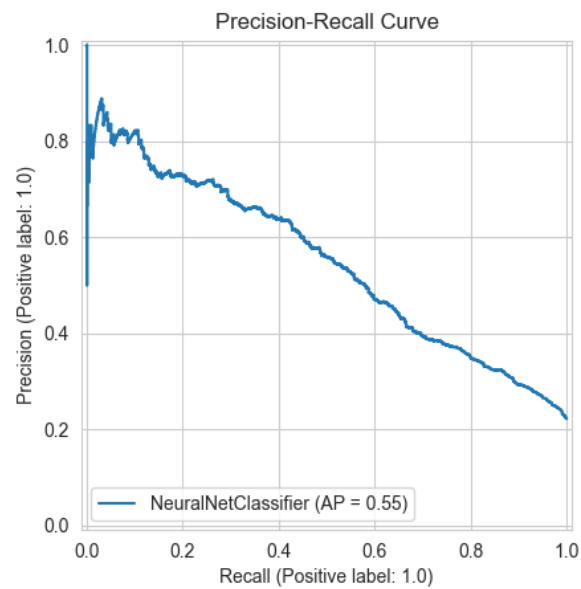
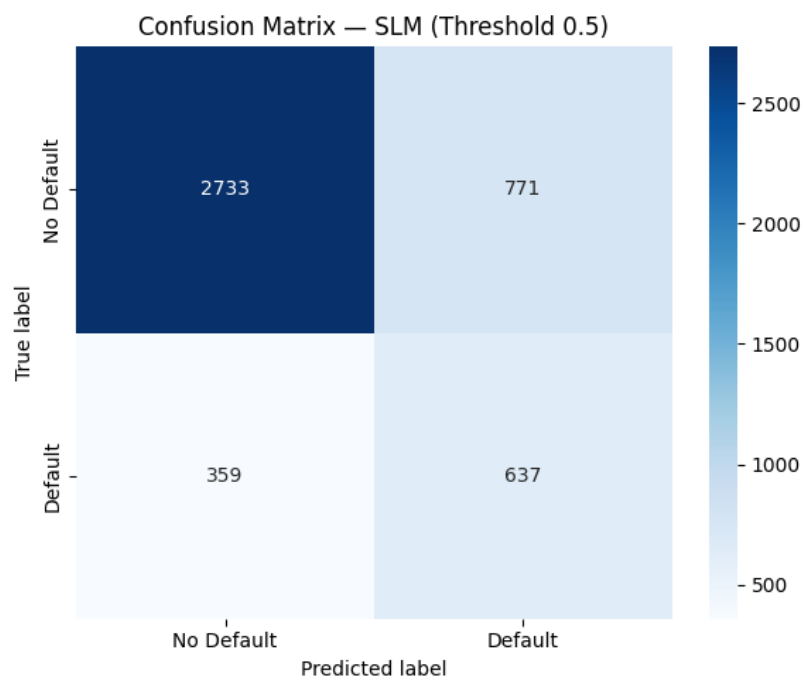
https://github.com/Noxren/FinanceAndAI_Project/tree/main

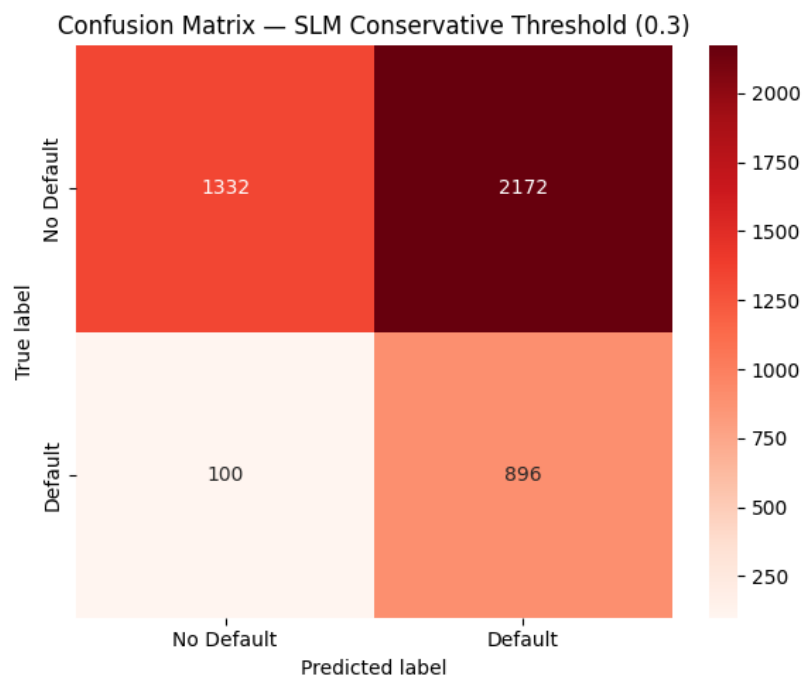
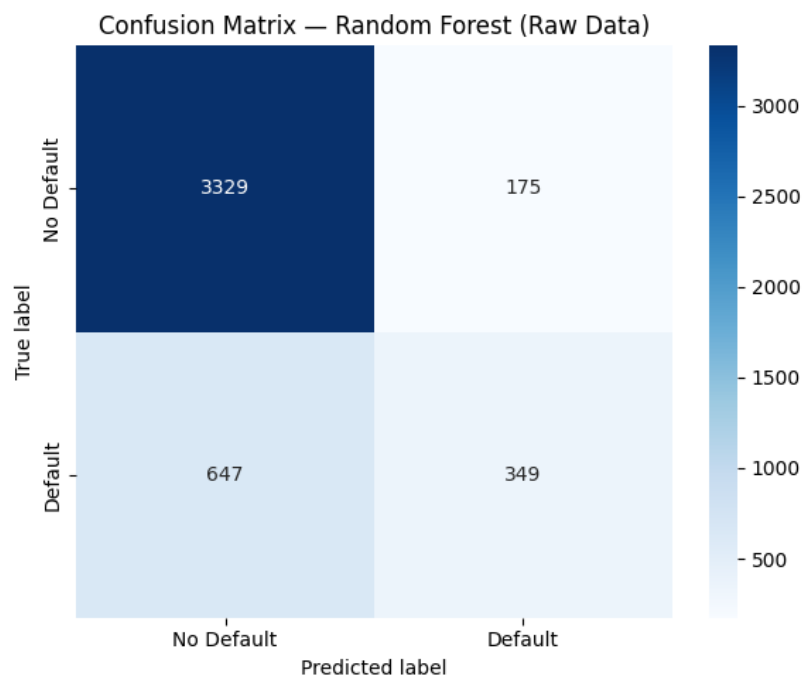
Exploratory Data Analysis



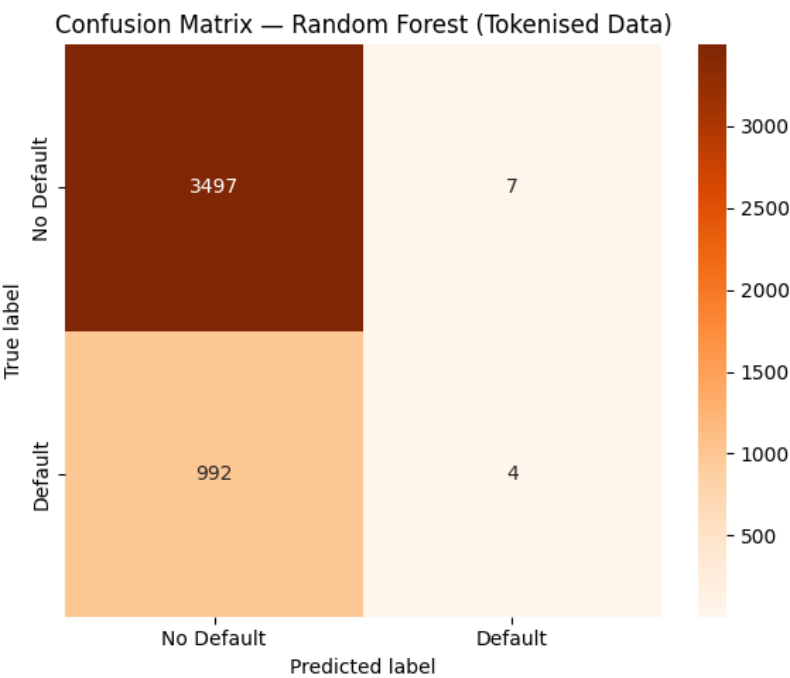
Training Loss Convergence - Small Transformer-based Model



Precision-Recall Curve - Small Transformer-based Model**Confusion Matrix - Small Transformer-based Model (Threshold 0.5)**

Confusion Matrix - Small Transformer-based Model (Threshold 0.3)**Confusion Matrix - Random Forest Model (Tabular Data)**

Confusion Matrix - Random Forest Model (Tokenised Data)



Feature Importance - Random Forest Model (Tokenised Data)

